https://github.com/jambao24/VineMapper-jambao24/blob/main/projects/East_Southeast_Asian_Groups_per_county/

using the original as a template
https://github.com/winstonhoyle/VineMapper/blob/main/projects/ethnicity/East_Asian_Groups_Per_County/FormatData.ipynb

2026.01.09- downloading cb_2024_us_county_500k.zip from here https://catalog.data.gov/dataset/2024-cartographic-boundary-file-shp-county-and-equivalent-for-united-states-1-500000

actually this may be more accurate? https://www2.census.gov/geo/tiger/GENZ2024/shp/

```
import requests
import geopandas as gpd
import pandas as pd


###Open County data

from osgeo import gdal
gdal.SetConfigOption('SHAPE_RESTORE_SHX', 'YES')

# https://catalog.data.gov/dataset/2024-cartographic-boundary-file-shp-county-subdivision-for-united-states-1-500000
# https://stackoverflow.com/questions/61436956/set-shape-restore-shx-config-option-to-yes-to-restore-or-create-it
# requires both .shp and .shx files in file directory
file_path = "/content/cb_2024_us_county_500k.shp"
counties_gdf = gpd.read_file(file_path)
```

```
### Get Ethnic Data
r = requests.get("https://api.census.gov/data/2023/acs/acs5/groups/B02018.json")
columns_obj = r.json()
```

```
###Get columns to query and rename for later
columns = []
rename_vars = {}
variables = columns_obj["variables"]

for name, variable in list(variables.items()):
    v_split = variable["label"].split("!!")
    if len(v_split) < 3:
        continue

    if v_split[0] == "Estimate":
        label = v_split[-1]
        rename_vars[name] = label

    if (name.endswith("E") or name.endswith("M")) and v_split[-2] == "East Asian:":
        columns.append(name)

    if (name.endswith("E") or name.endswith("M")) and v_split[-2] == "Southeast Asian:":
        columns.append(name)
```

```
columns.append("GEO_ID")
columns_formatted = ",".join(columns)

response = requests.get(
    f"https://api.census.gov/data/2023/acs/acs5?get={columns_formatted}&for=county:*"
)
```

```
data = response.json()
columns = data[0]
```

```
rows = data[1:]
df = pd.DataFrame(rows, columns=columns)
```

```
estimate_cols = [col for col in df.columns if col.endswith("E")]

formtted_df = df[["GEO_ID", *estimate_cols]]
formtted_df[estimate_cols] = formtted_df[estimate_cols].astype(int)

formtted_df["most_common_ancestry_raw"] = formtted_df[estimate_cols].idxmax(axis=1)
```

```
/tmp/ipython-input-658663868.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  formtted_df[estimate_cols] = formtted_df[estimate_cols].astype(int)
/tmp/ipython-input-658663868.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  formtted_df["most_common_ancestry_raw"] = formtted_df[estimate_cols].idxmax(axis=1)
```

```
def check_margin_error(row) -> str:
    geo_id = row["GEO_ID"]
    ethnicity_col = row["most_common_ancestry_raw"]
    val = row[ethnicity_col]

    if not val:
        return None

    moe_col = ethnicity_col.replace("E", "M")
    moe_val = int(df[df["GEO_ID"] == geo_id][moe_col])

    rmoe_val = abs(moe_val / val)
    if rmoe_val < 0.50:
        return variables[ethnicity_col]["label"].split("!!")[-1]
    else:
        return None
```

```
formtted_df["most_common_ancestry"] = formtted_df.apply(
    lambda row: check_margin_error(row), axis=1
)
```

```
/tmp/ipython-input-1230492617.py:10: FutureWarning: Calling int on a single element Series is deprecated and will raise a Typ
  moe_val = int(df[df["GEO_ID"] == geo_id][moe_col])
/tmp/ipython-input-2401641906.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  formtted_df["most_common_ancestry"] = formtted_df.apply(
```

```
rename_vars["GEO_ID"] = "GEOIDFQ"
formtted_df = formtted_df.rename(columns=rename_vars)

print(formtted_df.columns)

'''
# https://stackoverflow.com/questions/48854943/how-can-i-download-a-pandas-dataframe-in-google-colab
from google.colab import files
formtted_df.to_csv('formtted_df.csv')
files.download('formtted_df.csv')
'''
```

```
Index(['GEOIDFQ', 'Singaporean', 'Thai', 'Malaysian', 'Mien', 'Vietnamese',
       'Filipino', 'Indonesian', 'Laotian', 'Burmese', 'Cambodian', 'Korean',
       'Mongolian', 'Hmong', 'Japanese', 'Other East Asian', 'Okinawan',
       'Taiwanese', 'Chinese, except Taiwanese', 'Other Southeast Asian',
       'most_common_ancestry_raw', 'most_common_ancestry'],
      dtype='object')
'\n# https://stackoverflow.com/questions/48854943/how-can-i-download-a-pandas-dataframe-in-google-colab\nfrom google.colab i
mport files\nformtted_df.to_csv('formtted_df.csv') \nfiles.download('formtted_df.csv')\n'
```

```
###Merge Data

#print(counties_gdf)
# https://geopandas.org/en/stable/docs/user_guide/io.html
counties_gdf_xls = gpd.read_file("cb_2024_us_county_500k.dbf")
print(counties_gdf_xls.columns)
print(formtted_df.columns)

gdf = counties_gdf_xls.merge(formtted_df, on="GEOIDFQ", how="inner")

print("post merge:")
print(gdf.columns)
#print(gdf.head())

'''
gdf = gdf.to_crs(9311)
gdf.to_file("data/EastSoutheast_Asian_Groups_Per_County.gpkg")
'''
gdf.groupby("most_common_ancestry").size().reset_index(name="COUNT").sort_values(
    "COUNT", ascending=False
)
```

```
Index(['STATEFP', 'COUNTYFP', 'COUNTYNS', 'GEOIDFQ', 'GEOID', 'NAME',
       'NAMELSAD', 'STUSPS', 'STATE_NAME', 'LSAD', 'ALAND', 'AWATER',
       'geometry'],
      dtype='object')
Index(['GEOIDFQ', 'Singaporean', 'Thai', 'Malaysian', 'Mien', 'Vietnamese',
       'Filipino', 'Indonesian', 'Laotian', 'Burmese', 'Cambodian', 'Korean',
       'Mongolian', 'Hmong', 'Japanese', 'Other East Asian', 'Okinawan',
       'Taiwanese', 'Chinese, except Taiwanese', 'Other Southeast Asian',
       'most_common_ancestry_raw', 'most_common_ancestry'],
      dtype='object')
post merge:
Index(['STATEFP', 'COUNTYFP', 'COUNTYNS', 'GEOIDFQ', 'GEOID', 'NAME',
       'NAMELSAD', 'STUSPS', 'STATE_NAME', 'LSAD', 'ALAND', 'AWATER',
       'geometry', 'Singaporean', 'Thai', 'Malaysian', 'Mien', 'Vietnamese',
       'Filipino', 'Indonesian', 'Laotian', 'Burmese', 'Cambodian', 'Korean',
       'Mongolian', 'Hmong', 'Japanese', 'Other East Asian', 'Okinawan',
       'Taiwanese', 'Chinese, except Taiwanese', 'Other Southeast Asian',
       'most_common_ancestry_raw', 'most_common_ancestry'],
      dtype='object')
```

1 to 10 of 10 entries Filter

index	most_common_ancestry	COUNT
3	Filipino	491
2	Chinese, except Taiwanese	303
9	Vietnamese	62
6	Korean	53
4	Hmong	42
5	Japanese	28
0	Burmese	20
7	Laotian	13
1	Cambodian	4
8	Thai	3

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

https://www.arcgis.com/apps/mapviewer/index.html?url=https://geo.dot.gov/server/rest/services/Hosted/County_cb_2018_us_state_500k/FeatureServer&source=sd this is pretty cool

https://catalog.data.gov/dataset/2024-cartographic-boundary-file-shp-county-and-equivalent-for-united-states-1-500000

2026.01.09 https://pdxedu.maps.arcgis.com/apps/mapviewer/index.html using 2019 login info to access ArcGIS and play with shp file... runtime issue in Google Colab is with the Key in the file I have here...

> KeyError: 'AFFGEOIDFQ'

just used the .dbf spreadsheet from the zip file that contains GEO_ID columns, reran that code snippet and got the following error:

> ValueError: Cannot transform naive geometries. Please set a crs on the object first.